

HyperDistillation - Benchmarking network for containers

Mentors

- Aditi Ahuja
- Animesh N D
- Anirudh Lakhotia
- Anirudh Rowjee
- Monish Moger
- Smaran Jawalkar
- Vishwas Rajashekar

Team

- Anirudh Sudhir
- Navneet Nayak

Introduction

Goal: Show the potential of using eBPF tools to track network usage without depending on kubernetes for network infrastructure

- There are many tools that track network usage through kubernetes but if the application does not use kubernetes, it becomes extremely hard to keep track of network usage and how API calls move from one application to another.
- This project will try to observe communication outside of kubernetes with a drop in docker container requiring no additional setup.

HyperDistillation utilises Inspektor Gadget, a set of tools and framework for data collection and system inspection on Kubernetes clusters and Linux hosts using eBPF. It traces network traffic between containers and hosts, outside of a Kubernetes cluster. This trace is visualised in the form of a dependency graph, with additional filtering capabilities.

Development Log

31 July - Wednesday

TODO

- Filter TCP events based on container names provided by the user, as well as regex matching
- Filter packets based on speed/latency etc...

Getting Container Names

- Initially, we looked into using the ‘container collection’ library provided by inspektor gadget to find the names of all currently active containers and use it to filter tcp events
- However, it did not fit our use case as it used the mount namespace inode ID to identify containers and the trace TCP gadget didn’t provide mount namespace information for the destination IP.
- This let us identify the source container name but not the destination container name
- Unfortunately we spent some time on this to no avail, but we went back to using the docker API directly to get container names.

Modularisation

We modularised the code to split the main functionalities of the tool and make them independent of each other to avoid the overhead of running the other functionalities when they are not required.

The three functionalities are:

- TCP Trace:
 - Uses the trace TCP tool to log all TCP connection events.
 - Uses the docker API to perform IP to container name matching
 - Stores all the logs in json format for future use / use by other modules
- Filtering
 - Converts the json logs back into event objects to start filtering
 - Can filter either by a list of container names or regex provided by the user from the command line or from other code.
- Graphing
 - Takes the event logs either from the json log file and converts it to event objects or takes the event logs from memory directly from the Filtering module
 - Creates a .gv file from the provided event logs

- 'make graph' can be run to visualise the .gv file by converting it to an svg.

All three functionalities are packaged into one binary and can be called separately each time the binary is executed or all at once. This means no re-compilation.

Tests conducted

https://github.com/PES-Innovation-Lab/hyperdistillation/tree/main/tests/graph_tcp_filter

- We conducted several tests between containers and were able to perform multiple name based filtering and regex filtering
- The below graph shows the complete TCP trace without any filters

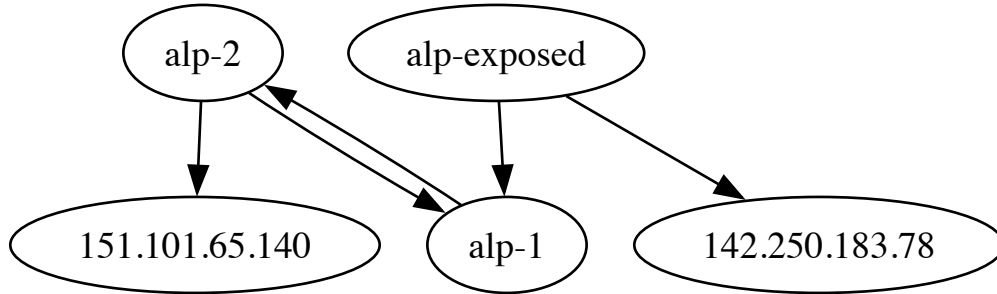


Figure 1: no_filter.svg

- The below graph contains TCP events filtered for the “alp-1” and “alp-exposed” container from the same trace log

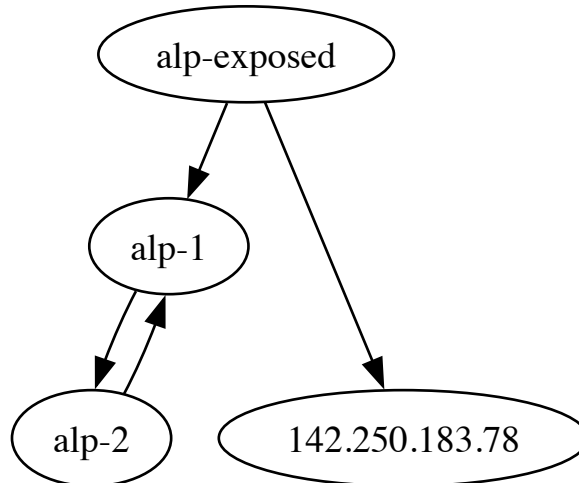


Figure 2: filter_name_alp-1_alp-exposed.svg

- We can filter containers based on regex as well, allowing you to narrow down on containers you want to graph having to explicitly name all of them
 - For example we can pass the regex “2” which would narrow down all containers with “2” in the name, the regex matching ignores anything other than those container names.

Container Label / Parameter Filtering

The docker API gives us a lot of information about each container, we weren't sure on how to go about making a general framework that would let you filter containers based on any parameter, while also being

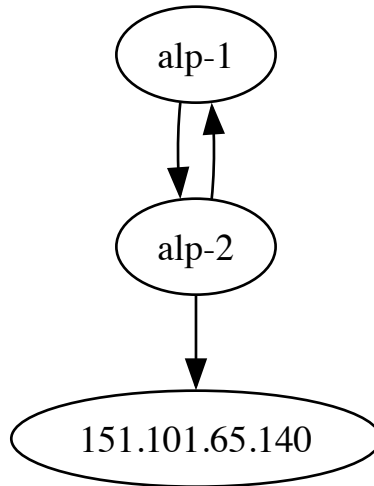


Figure 3: filter_regex_2.svg

efficient, hence we worked on filtering based on Container Name.

- When working on filtering based on Container Names, we realised that the matching code can need to be pretty specific based on what your are trying to filter for, making it harder to make a truly general filter that can filter containers based on any label provided by the docker API.
- An approach to this is to find all the fields that are important and try to write general filter code for as many as possible, while writing specific filter functions for other parameters that require it.
- An alternate idea was to collect all the data provided by the docker API for each TCP event, which would contain all the metadata with respect to the containers and then perform a regex match on all fields of this metadata struct
 - This would not be the best solution as it performs a generic regex match and might throw up incorrect results, while also being inefficient with respect to storage of metadata structs in memory as well as pattern matching all of the fields.

Packet Filtering and UDP

Due to the limitations of the trace TCP tool:

- Not being able to trace the packets themselves, we could not implement packet filtering based on parameters like speed, latency etc.
- Not being able to trace UDP socket operations, we could not implement UDP tracing as well.

30 July - Tuesday

TODO

Use the docker API to get all container names and create a dependency graph using container names, assign all other socket events such that they originate from the host.

Container Names

Using the docker API, we were able to get the names and IP's of all docker containers currently active as well as other metadata. We map IP addresses to docker containers and add metadata to the event object we get from the eBPF gadget.

- The metadata enriched events are used to create nodes and edges in the graph.
- All TCP connection events that cannot be traced back to a container are treated as host events
- TCP connection events by containers to remote hosts are considered in the graph, whereas TCP connection events by the host to remote hosts are ignored to avoid cluttering the graph, they can easily be enabled.
- Currently the graph does not prevent cycles, but it can also be easily changed to do so.
 - If container 1 sends a request to container 2 at a certain instance and vice-versa at another instance, tracing both these events involves creation of a cycle

A dependency (an edge) is only created when a container performs a connection request with another container or a remote host, not when socket ‘accept’ or ‘close’ operations occur.

Tests Conducted

Link to test results: https://github.com/PES-Innovation-Lab/hyperdistillation/tree/main/tests/graph_trace_tcp_docker_api

Since we are using the docker API, it is able to trace all currently running containers with their IP’s even when they are on a different virtual network.

In the below example, pinger and pinger2 are containers on the virtual bridge, and net_c and net_c2 are two nginx servers on a user defined virtual network.

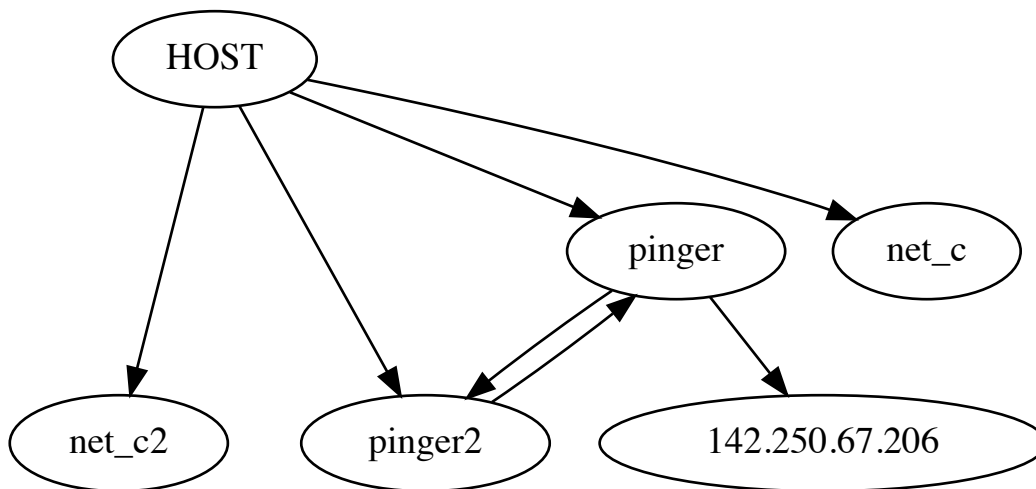


Figure 4: graph_final.gv.svg

*The ip addresses represent remote hosts, in this case ‘google.com’

29 July - Monday

TODO

Parse tcptrace into a dag

DAG Based on TCP Trace logs

Parsing events produced by the TCP tracer and create a DAG

- Each node is represented by an IP

- We created edges from the source to the destination IP for each TCP trace event. The DAG was produced using the <https://github.com/dominikbraun/graph> library.
- Some addresses like 127.0.0.1(localhost), ::1(docker proxy) connect to themselves hence we further differentiate them with the port number.

Tests conducted

The tests include

- Sending requests from docker containers to other containers hosting python web servers
- Sending curl requests from the containers to google.com
- Sending requests from the host to the containers using the docker virtual bridge IPs (172.17.0.x)
- Sending requests from the host to the containers via localhost to port-mapped containers (involves docker-proxy, which proxies requests from the host port to the container's port)
- Sending requests from the container to the host running a web server using the host IP
- Sent requests to invalid IPs (localhost port 8000 on containers which did not have a web server running) and filtered them from the graph

Results

- The trace tcp tool was able to log all “connect”, “accept” and “close” events which were used to generate the DAG.
- The tool treats requests from host as requests from the gateway IP, which here was 172.17.0.1

The trace logs and graphs can be viewed here: https://github.com/PES-Innovation-Lab/hyperdistillation/tree/main/tests/graph_trace_tcp

- Sample generated graphs

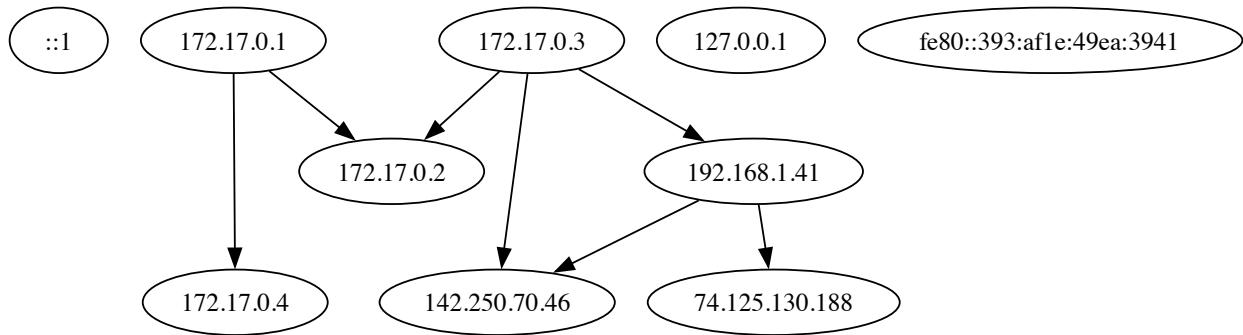


Figure 5: host_container_test_2.svg

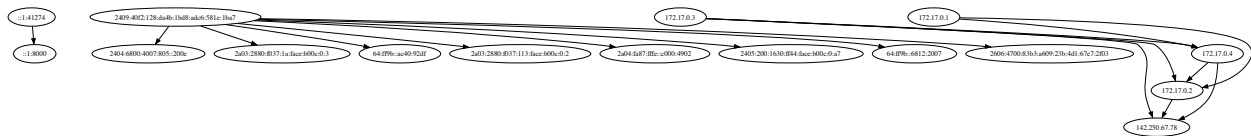


Figure 6: host_container_test_4.svg

27 July - Saturday

TODO

Why tcptrace/dnstrace can see more than network trace? Parsing the tcptrace info into a dag

To answer the question:

Why tcptrace/dnstrace can see more than network trace?

tcp trace is attached to many system hooks including “tcp_v4_connect” etc.. This kernel function gets called to establish a tcp connection with a remote host and has access to socket information. The eBPF program is extracting the src and dest ip’s from the socket struct.

trace network is attached to the “socket1” hook, and has access to ‘skb’ (or socket buffer) which the Linux Kernel uses to store network packet information. The destination ip is extracted from the ip header of a packet.

As for why tcp trace can list both the source and destination ip’s and trace network can only list the dest ip - Even though a packet’s ip header contains information about both the src and dest ip’s, the eBPF program that traces network simply doesn’t return the src ip. You can see in the next section that we were able to log the src ip from the eBPF file with some modifications.

Getting trace network to list both source and destination ip’s

We found this in one of the inspektor gadget blogs:

The bpf2go model requires checking the compiled eBPF binaries into the repository **as** they are embedded into the binary. **Otherwise**, the created Golang packages **for** the gadgets won’t be usable **as** the eBPF code will be missing.

This means that BPF code is precompiled and embedded into the inspektor-gadget go package, hence if we want to change any of the eBPF code then we need to recompile the eBPF code in your go packages folder using ‘go generate’ in a privileged context.

On changing the trace network eBPF code and regenerating the eBPF object files, we are able to get “trace network” to print both the source and destination ip of a packet into the trace pipe, the IPs are displayed in raw format. In order to parse the IPs into human readable format, we need to work further towards understanding how exactly the go interface is getting the eBPF map information and parsing it.

```
7343, Dest ip: 889192575
code-6925 [002] ..s21 7879.052993: bpf_trace_printk: Src ip: 167
7343, Dest ip: 889192575
code-6925 [002] b..11 7879.053039: bpf_trace_printk: Src ip: 167
7343, Dest ip: 889192575
code-6925 [002] ..s21 7879.053052: bpf_trace_printk: Src ip: 167
7343, Dest ip: 889192575
systemd-resolve-819 [003] b..21 7879.054237: bpf_trace_printk: Src ip: 117
549248, Dest ip: 16885952
code-6902 [005] b..21 7879.060011: bpf_trace_printk: Src ip: 117
549248, Dest ip: 397262100
```

Figure 7: Untitled

Generation of Directed Acyclic Graphs (Work in-progress)

- When used with kubectrl gadget or the ig command line tool, the output can be saved as JSON formatted data. However, the trace APIs return data as a struct each time an event is triggered, requiring another solution
- Plan of action to generate DAGs
 - On event trigger during tracing, save the struct in a slice
 - Listen for a SIGINT, once triggered, stop tracing and create the DAG from the slice using one of the following packages
 - * <https://github.com/gonum/gonum>
 - * <https://github.com/dominikbraun/graph>
 - * Both of these packages can be used to create a visualisation of the graph

Things to look into

- The inspektor-gadget repo has a Makefile with an “ebpf-objects” target to compile the eBPF code of the gadgets, which make help in the modification of gadgets if required

```
make ebpf-objects
```
- Look into how the Go interface obtains the eBPF map info so that
 - Parse the raw IP into a readable address, which does not seem to be allowed by the kernel
 - IPs and other useful data can be directly parsed from Go code rather than modifying the eBPF gadgets

26 July - Friday

TODO

Figure out when docker proxy triggered by Friday eod Figure out trace agent if possible by EOD Friday, if not possible stick with tcp tracing and see if there's a tracer for other events like udp and icmp, etc. Stretch task, if this can start this week that'd be great but it's okay if it doesn't. Essentially try parsing the trace data and create a dag. (Ig might have a library in this already so check there first)

Tasks completed

Attempted to visualise the “trace network” API

- Found an example on the usage of the “trace dns” gadget (<https://github.com/inspektor-gadget/inspektor-gadget/blob/main/examples/builtin-gadgets/basic/trace/dns/dns.go>)
 - The example did not work out of the box, we had to add “tracer.RunWorkaround()” after attaching the eventCallback function and print relevant data
- We used this example to run “trace network”, which had a similar set of APIs
 - The “trace network” gadget was able to trace only certain TCP and UDP connections which were of “HOST” and “OUTGOING” type
- The docker-proxy is used to receive connections from the host and forward it to the container, usually when port-mapping has been setup (<https://windsock.io/the-docker-proxy/>)
- To perform these tests, we used the “trace tcp” gadget as to identify the processes
 - https://github.com/PES-Innovation-Lab/hyperdistillation/blob/main/trace_tcp_output/docker_proxy_host_to_docker_ip.txt
 - * When the IP assigned by the virtual network bridge is used to send a curl request to the container which has been port-mapped, the docker-proxy is **not** present in the trace
 - https://github.com/PES-Innovation-Lab/hyperdistillation/blob/main/trace_tcp_output/docker_proxy_host_to_localhost.txt

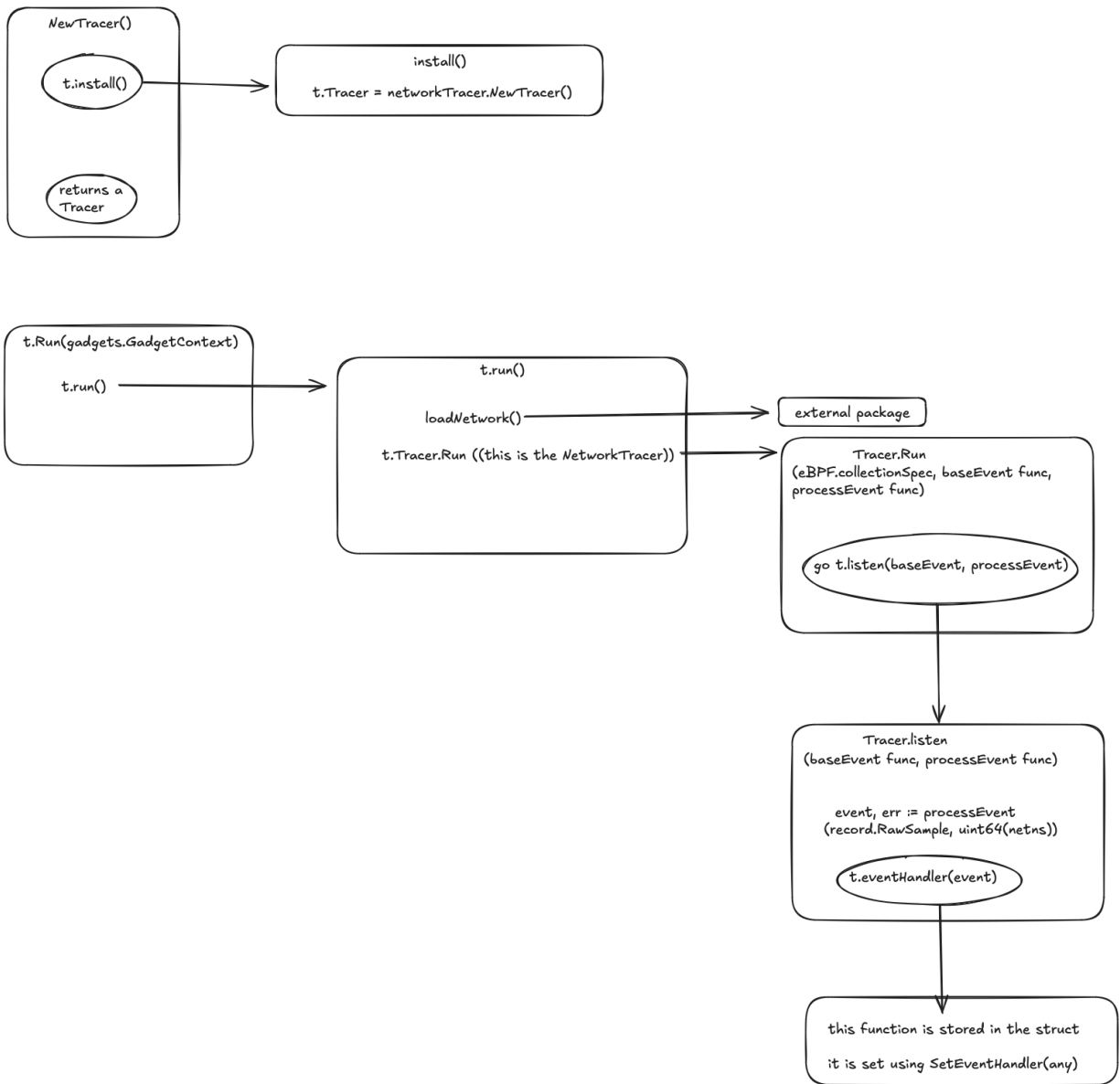


Figure 8: Untitled

- * When localhost is used to send a curl request to the container which has been port-mapped, the docker-proxy is present in the trace

Tests conducted

https://github.com/PES-Innovation-Lab/hyperdistillation/tree/main/trace_network_output

- curl google.com from host
 - Only the “OUTGOING” DNS (UDP) and TCP requests were traced

```
// DNS Server for the host: 103.57.86.36
// google.com resolves to 142.250.183.78
```

```
Timestamp: 2024-07-26T18:48:37.946988795+05:30, Type: normal, Mount Ns ID: 0, Network Ns ID: 402653
Packet Type: OUTGOING, Protocol: UDP, Port: 53, Destination Endpoint: {103.57.86.36 4 map[]}
```

```
Timestamp: 2024-07-26T18:48:37.947094675+05:30, Type: normal, Mount Ns ID: 0, Network Ns ID: 402653
Packet Type: OUTGOING, Protocol: UDP, Port: 53, Destination Endpoint: {103.57.86.36 4 map[]}
```

```
Timestamp: 2024-07-26T18:48:37.973053741+05:30, Type: normal, Mount Ns ID: 0, Network Ns ID: 402653
Packet Type: OUTGOING, Protocol: UDP, Port: 53, Destination Endpoint: {103.57.86.36 4 map[]}
```

```
Timestamp: 2024-07-26T18:48:37.993254322+05:30, Type: normal, Mount Ns ID: 0, Network Ns ID: 402653
Packet Type: OUTGOING, Protocol: TCP, Port: 80, Destination Endpoint: {142.250.183.78 4 map[]}
```

- curl google.com from a container
 - The “OUTGOING” and “HOST” DNS and the “OUTGOING” TCP requests were traced
- host-host and container-container netcat queries using TCP and UDP did not give useful results
- curl localhost:8000 from host to a container with a python-server and port-mapping gave “OUTGOING” and “HOST” TCP requests on the host and “OUTGOING” TCP requests on the container
- curl from host to container using the virtual IP (172..) gave an “OUTGOING” TCP request on the container

Looking into the eBPF code

- All the types of packets from the linux kernel, this states what ‘HOST’ packet and all other types of packets mean

```
#define PACKET_HOST      0      /* To us      */
#define PACKET_BROADCAST 1      /* To all     */
#define PACKET_MULTICAST 2      /* To group   */
#define PACKET_OTHERHOST 3      /* To someone else */
#define PACKET_OUTGOING  4      /* Outgoing of any type */
#define PACKET_LOOPBACK  5      /* MC/BRD frame looped back */
#define PACKET_USER      6      /* To user space */
#define PACKET_KERNEL     7      /* To kernel space */
/* Unused, PACKET_FASTROUTE and PACKET_LOOPBACK are invisible to user space */
#define PACKET_FASTROUTE  6      /* Fastrouted frame */
```

- The eBPF code skips tracing all types of packets other than ‘HOST’ and ‘OUTGOING’ and returns early, unfortunately when commenting this if statement, the tracer was still not tracing any packet of the other types, although this may have to do with an insufficient test environment.
- Looking more into the eBPF code it also does not create events for packets of any protocol other than TCP and UDP, so if we can figure out how to parse other packet’s information, we could trace those packets as well.
- This was in one of the examples provided, the trace network api requires a tracer to attach to a process, meaning the tracer was only tracing network traffic associated with one network namespace, specifically

```
// Skip multicast, broadcast, forwarding...
if (skb->pkt_type != PACKET_HOST && skb->pkt_type != PACKET_OUTGOING)
    return 0;
```

Figure 9: Untitled

```
} else {
    // Skip packets with IP protocol other than TCP/UDP.
    return 0;
}
```

Figure 10: Untitled

“4026531840”. We found that on both our machines, it was tracing network packets only associated with this network namespace. If we can find a way to trace packets of all network namespaces or attach a tracer to at least one process on each netns, we could presumably extract a lot more network information.

```
// The tracer has to be attached. The DNS packets will be traced on
// the network namespace of pid.
pid := uint32(os.Getpid())
if err := tracer.Attach(pid); err != nil {
```

Figure 11: Untitled

- We also found this in the go tracer code, where the go code is replacing the eBPF map defined in eBPF code written in C with the tracer object’s map containing information about current network names-

```
opts := ebpf.CollectionOptions{
    MapReplacements: map[string]*ebpf.Map{
        "tail_call": t.dispatcherMap,
    },
}
```

pace??, we’ll have to look into this further.

Things to Look into:

- The interfacing between the C and Go code, specifically look more into how the code is structured and which functions call which, and how exactly the Go code interacts with the eBPF code written in C. (like the Map replacing(?) code)
- Look more into eBPF programs as well, learn about how they work and the current eBPF setup so that it can be customized if needed.
- Trace all “HOST” and “OUTGOING” traffic as well as host-host, container-container and host-container netcat queries using TCP and UDP
- Understand and discover the various network namespaces on a system at a particular instance

```
lsns --type=net
readlink /proc/<pid>/ns/net
```

25 July - Thursday

TODO:

Figure out the usage of the go script to filter host events, and container events for the current setup of nginx, ping and host interactions by Friday, share whatever progress made and report by EOD tomorrow

Tasks Completed

Using the Inspektor Gadget control plane APIs in the application: <https://www.inspektor-gadget.io/blog/2022/09/using-inspektor-gadget-from-golang-applications/>

- The Inspector Gadget application consists of the eBPF-based gadgets which run in the kernel space and a control plane for those programs written in Go which runs in the userspace
- This architecture allows external programs to utilise the various gadgets for tracing and observability
- We compiled and ran the example mentioned in the blog to trace all processes created on the host
 - It was able to successfully trace process creation and provided access to various parameters such as PID.
- We then attempted to use the “trace network” gadget and create a new “Tracer”. However, the API isn’t well documented and we looked into the source code to understand its usage.
- We weren’t able to attach event handlers and trace network calls, so we tried using the “trace tcp” gadget, which can monitor “connect”, “accept” and “close” events related to TCP connections. This gadget has a similar API as “trace exec”
- We were able to trace all the TCP connections made by
 - the host system with external services (background communication)
 - container and host communication (The container ran a python server and we performed a curl request from the host machine)
 - inter-container communication (Another container curled the python container)
 - Remote device-to-container communication (exposing a port on the container and sending a curl request from a remote device)
 - Container-to-remote device communication (Sending a curl request from the container to another device on the network)
- Note: In case of remote connections, only the “connect”/”accept” message and “close” messages on the host can be traced by the “trace tcp” gadget

Tests Conducted

- Socket information for two docker containers, one hosting a python server and another pinging it, both containers on the same host machine

```
&{{{{      } { {  map[] } false}} 2024-07-25T16:34:42.300358864+05:30 normal } {4026532895} connect 68327
&{{{{      } { {  map[] } false}} 2024-07-25T16:34:42.300516498+05:30 normal } {4026533131} accept 67956 (
&{{{{      } { {  map[] } false}} 2024-07-25T16:34:42.302047716+05:30 normal } {4026533131} close 67956 0
&{{{{      } { {  map[] } false}} 2024-07-25T16:34:42.302193686+05:30 normal } {4026532895} close 68327 0
```

- Pinging the containerised python http server from a remote machine

```
&{{{{      } { {  map[] } false}} 2024-07-25T16:50:46.264195171+05:30 normal } {4026532979} accept 69377 0
&{{{{      } { {  map[] } false}} 2024-07-25T16:50:46.268291620+05:30 normal } {4026532979} close 69377 0
```

- Pinging the containerised python http server directly from the same host machine, in this case docker-proxy was activated

Services

Services provide a stable endpoint (IP address and DNS name) for accessing pods. Services abstract pod IP addresses, allowing for loose coupling between microservices.

Network Policies

Network Policies control the traffic flow at the IP address or port level. They specify how groups of pods are allowed to communicate with each other and other network endpoints.

3. Core Networking Components

a. Container Network Interface (CNI)

Kubernetes uses the CNI to manage network resources. CNI is a standard interface designed to configure network interfaces in Linux containers. It enables pluggable networking backends.

b. Kube-proxy

Kube-proxy is a network proxy that runs on each node in the cluster, maintaining network rules on nodes. It handles the routing of traffic directed to services and is responsible for implementing service abstractions (ClusterIP, NodePort, and LoadBalancer).

4. Types of Services

a. ClusterIP

ClusterIP is the default type of service, exposing the service on a cluster-internal IP. This IP can only be accessed from within the cluster.

b. NodePort

NodePort exposes the service on each node's IP at a static port. A NodePort service makes a ClusterIP service accessible externally by routing requests through the node's IP and port.

c. LoadBalancer

LoadBalancer creates an external load balancer (if supported by the cloud provider) and assigns a fixed, external IP to the service. It's an extension of NodePort and ClusterIP.

5. Pod-to-Pod Communication

Pods communicate with each other directly using their IP addresses. The network plugin configured with CNI ensures that every pod gets a unique IP address. This direct communication avoids the complexity of NAT and simplifies network policies and service discovery.

6. Service Discovery

Service discovery in Kubernetes is typically handled by two primary methods:

a. Environment Variables

Kubernetes injects environment variables into pods that contain the service's IP and port information.

b. DNS

Kubernetes includes a DNS server (CoreDNS) that provides DNS resolution for services and pods. It allows services to be accessible by their DNS names, e.g., my-service.my-namespace.svc.cluster.local.

7. Network Policies

Network policies specify how groups of pods are allowed to communicate with each other and with other network endpoints. They use labels to select pods and define rules to allow or deny traffic to/from those pods.

8. Ingress

Ingress is a collection of rules that allow inbound connections to reach the cluster services. An Ingress controller (like NGINX, Traefik) is responsible for fulfilling the Ingress rules and providing load balancing, SSL termination, and name-based virtual hosting.

9. Egress

Egress policies control the traffic leaving the cluster. Kubernetes does not support egress policies natively, but they can be implemented through third-party plugins or custom configurations.

Why is traffic observation easier on kubernetes

1. Sidecar Containers:

Service Meshes (e.g., Istio, Linkerd): These can be deployed as sidecar containers alongside application containers, intercepting and managing traffic. They provide detailed metrics, tracing, and observability features that are more advanced than what is typically available with Docker alone.

1. Namespace Isolation:

Namespaces: Kubernetes namespaces allow for logical separation of resources, making it easier to observe and manage traffic within different segments of the application.

1. CRDs

Kubernetes allows the creation of Custom Resource using Custom Resource Definitions(CRDs) (Inspektor Gadget provides tracer CRDs which create tracers that install eBPF programs)

1. Built-in Metrics and Dashboards:

Kubernetes Dashboard: Provides a web-based interface to monitor the cluster, including traffic metrics.

Metrics Server: Gathers resource usage data (e.g., CPU, memory) that can indirectly help in observing traffic by correlating resource usage with traffic patterns.

1. Architecture and control plane:

Control Plane, Services, Nodes, Pods, Individual containers

In contrast, Docker alone lacks these higher-level abstractions and integrations. While observing traffic in Docker, it requires more manual setup and lacks the out-of-the-box solutions available in a Kubernetes environment.

Doubt:

- Cilium - Networking/CNI plugin
 - Uses eBPF (Extended Berkeley Packet Filter) technology for high-performance networking and security.
 - Provides advanced network visibility and security features.

A gadget is composed by eBPF programs and maps that are compiled into an eBPF ELF object. According to the names of the maps and programs, and the types used, Inspektor Gadget is able to understand how to parse, enrich and print information from the gadget. Inspektor Gadget requires the ELF file to have sections named according to the convention defined in libbpf and *cilium/ebpf*. (<https://www.inspektor-gadget.io/docs/v0.30.0/design/002-containerized-gadgets/>)

How ig observes traffic

<https://github.com/inspektor-gadget/inspektor-gadget/blob/main/docs%2Fig.md>

<https://www.brendangregg.com/blog/2019-01-01/learn-ebpf-tracing.html>

- It uses eBPF as its underlying core technology

- To trace pods, ig attaches eBPF programs to kernel functions and the kernel will run them always when the functions are executed. Therefore, the eBPF programs need to detect if the syscall that triggered the function comes from a container that ig should trace. To do that the program looks up the current `mount` namespace id in a eBPF map containing the list of containers to trace, if it's not found the program exits early. Finally, the eBPF program gathers the information to trace, e.g., syscall parameters, and writes them to a ring buffer or eBPF map and ig reads it
- The **Gadget Tracer Manager** keeps a list of running gadgets and containers. Each running gadget has an associated eBPF map that is filled with the `mount` namespace identifiers of the containers to be traced according to the namespace, labels, pod name, etc. parameters passed to the gadget. The **Gadget Tracer Manager** knows about the current running containers thanks to `runc-fanotify` which adds or removes container to the **Gadget Tracer Manager** collection.

Using Inspektor Gadget from Golang Applications

<https://www.inspektor-gadget.io/blog/2022/09/using-inspektor-gadget-from-golang-applications/>

- Could trace all events from host by using the appropriate API directly